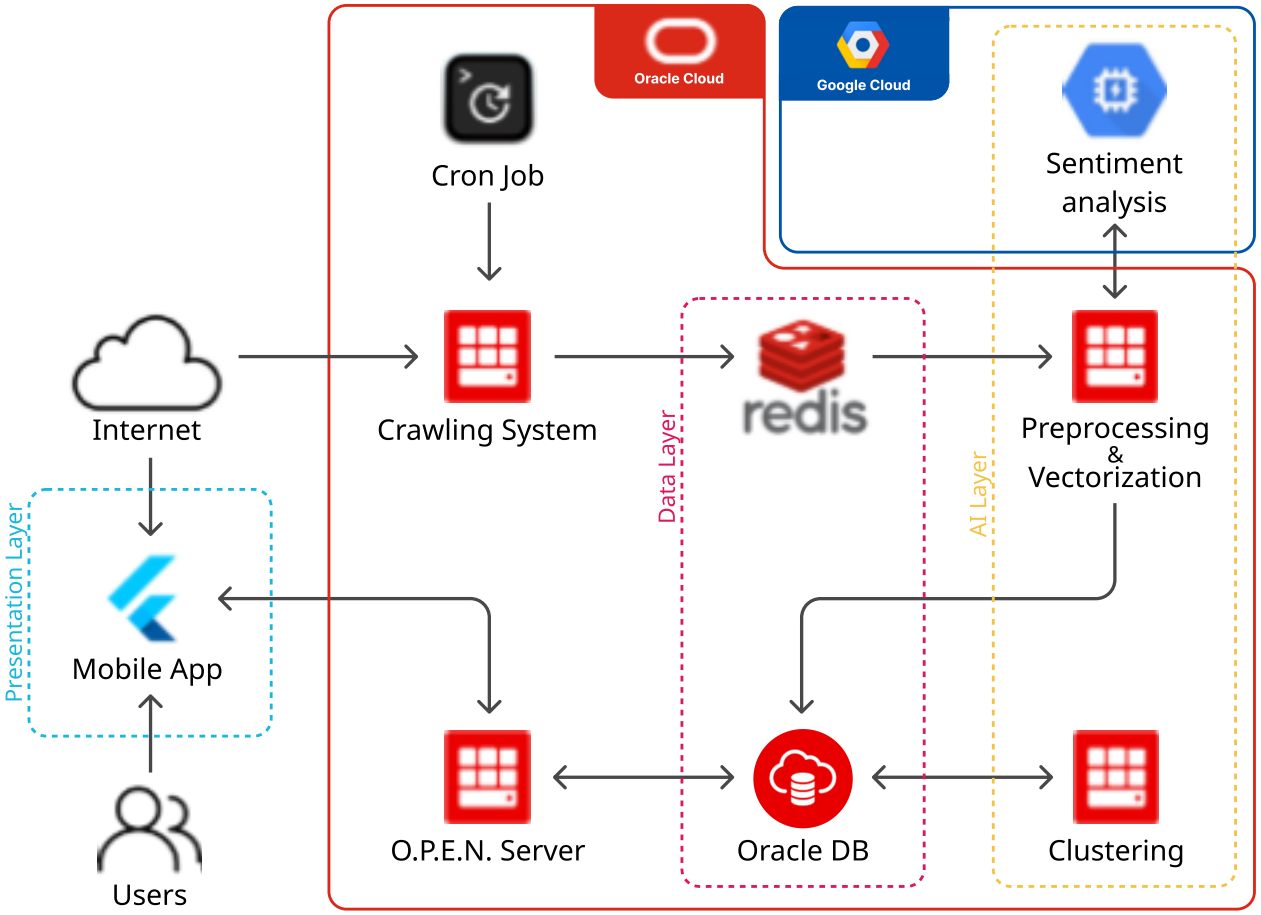
Architecture



벡터화 시스템

💡 벡터화 시스템 구현 과정에서의 고민과 문제

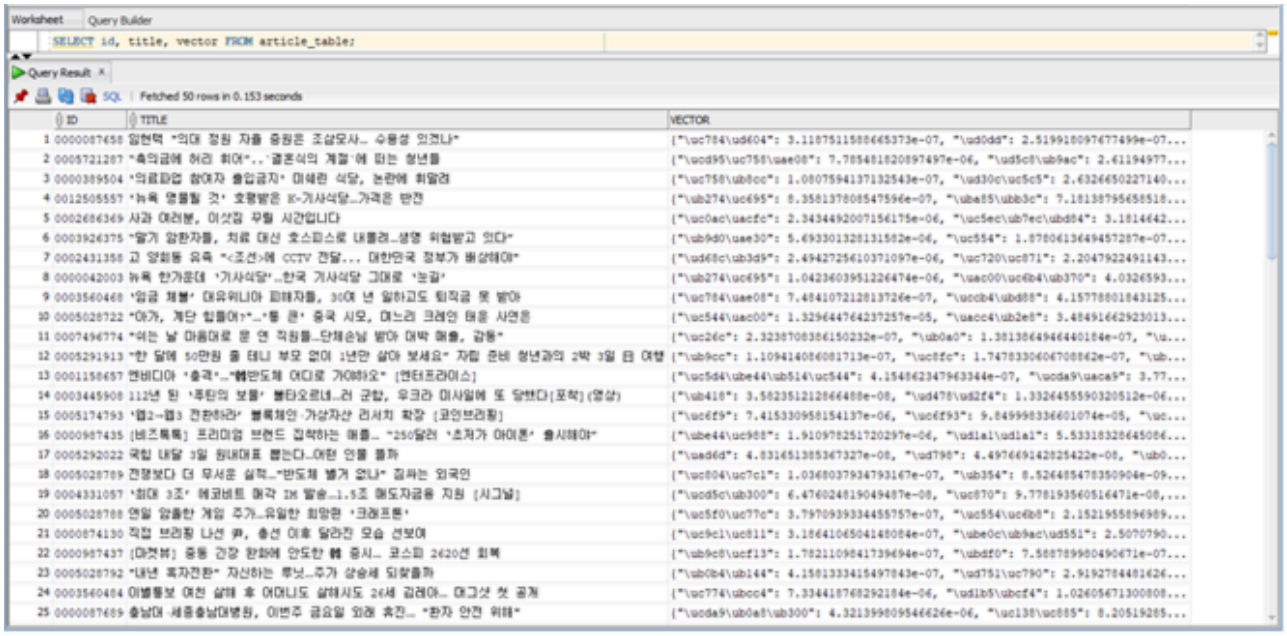
- 각각의 기사에 대해 벡터화를 진행할 때, 벡터의 크기가 너무 방대해지는 문제 발생
- 기존의 TF-IDF만으로 핵심 토큰을 추출해 내는 부분에서 문제점을 느꼈음
- 자주 반복되는 단어지만 핵심인 경우도 고려해야 할 필요성을 느꼈음 (ex, 날씨예보 관련 뉴스 기사에서 날씨는 핵심 용어지만 기사 내에서 이용 빈도가 너무 높아 가중치가 낮았음)

🛠️ 해결 방안

- 카테고리별로 분모를 설정해서 TF-IDF 벡터화를 진행했다. 예를 들어 과학 분야의 기사에는, 정치인의 이름이 주제의 핵심일 가능성은 적다고 판단하였기 때문에
- 즉, 우리가 설정한 5개의 분야에 대해서 global dictionary를 만들고 이를 이용해서 idf를 연산
-

✅ 결과 및 성과

- 벡터화 결과가 성공적으로 DB에 저장되어 있는 모습: {'토큰': '값', ~} 형태



클러스터링 시스템

💡 고민과 문제

- 벡터화된 데이터와 코사인 유사도를 활용하여 비슷한 군집끼리 클러스터를 형성하는 것을 목표로 설정
- hdbscan,hierarchical clustering등 다양한 방법이 존재했고 어느것을 사용할지 직접 수행 후 선택
    - hdbscan 방식으로 클러스터링을 수행했지만 원하는 군집들이 형성되지 않았음

🛠️ 해결 방안

- hdbscan의 문제점은 벡터가 너무 많이 존재하기 때문에, 즉 차원이 너무 많아서 모두가 개별 클러스터로 군집이 형성되는 문제가 있었음
    - 이를 해결하기 위해 차원 감소 기법을 적용했지만 오히려 해당 뉴스 기사 데이터의 식별성이 감소하여 군집화는 되었지만 원하는 정도의 클러스터 수가 나오지 않았음
- 이후 hierarchical clustering을 코사인 유사도를 활용하여 수행했을 때 만든 테스트 셋과 완전히 일치하는 군집이 형성되었고 이에 계층적 클러스터링 방식을 채택
    - 코사인 유사도를 활용하여 hierarchical clustering 수행

```
# 코사인 유사도 계산
cosine_sim_sparse = cosine_similarity(tfidf_matrix_sparse)

print(cosine_sim_sparse)
print(np.shape(cosine_sim_sparse))
```

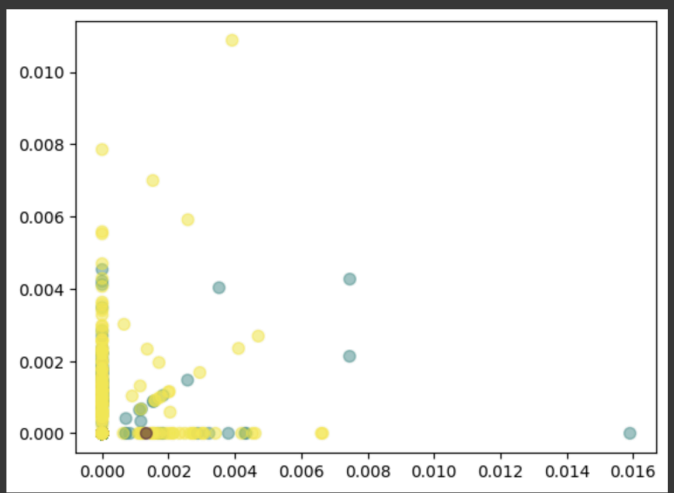
```
[[1.         0.01049017 0.89455853 ... 0.00466175 0.03593722 0.02043861]
 [0.01049017 1.         0.01123391 ... 0.00572088 0.02099814 0.01402911]
 [0.89455853 0.01123391 1.         ... 0.00541302 0.03236554 0.01752331]
 ...
 [0.00466175 0.00572088 0.00541302 ... 1.         0.00597598 0.01294041]
 [0.03593722 0.02099814 0.03236554 ... 0.00597598 1.         0.01488566]
 [0.02043861 0.01402911 0.01752331 ... 0.01294041 0.01488566 1.        ]]
(1200, 1200)
```

✅ 결과 및 성과

- hdbscan으로 군집이 제대로 형성되지 않은 모습

```
import matplotlib.pyplot as plt

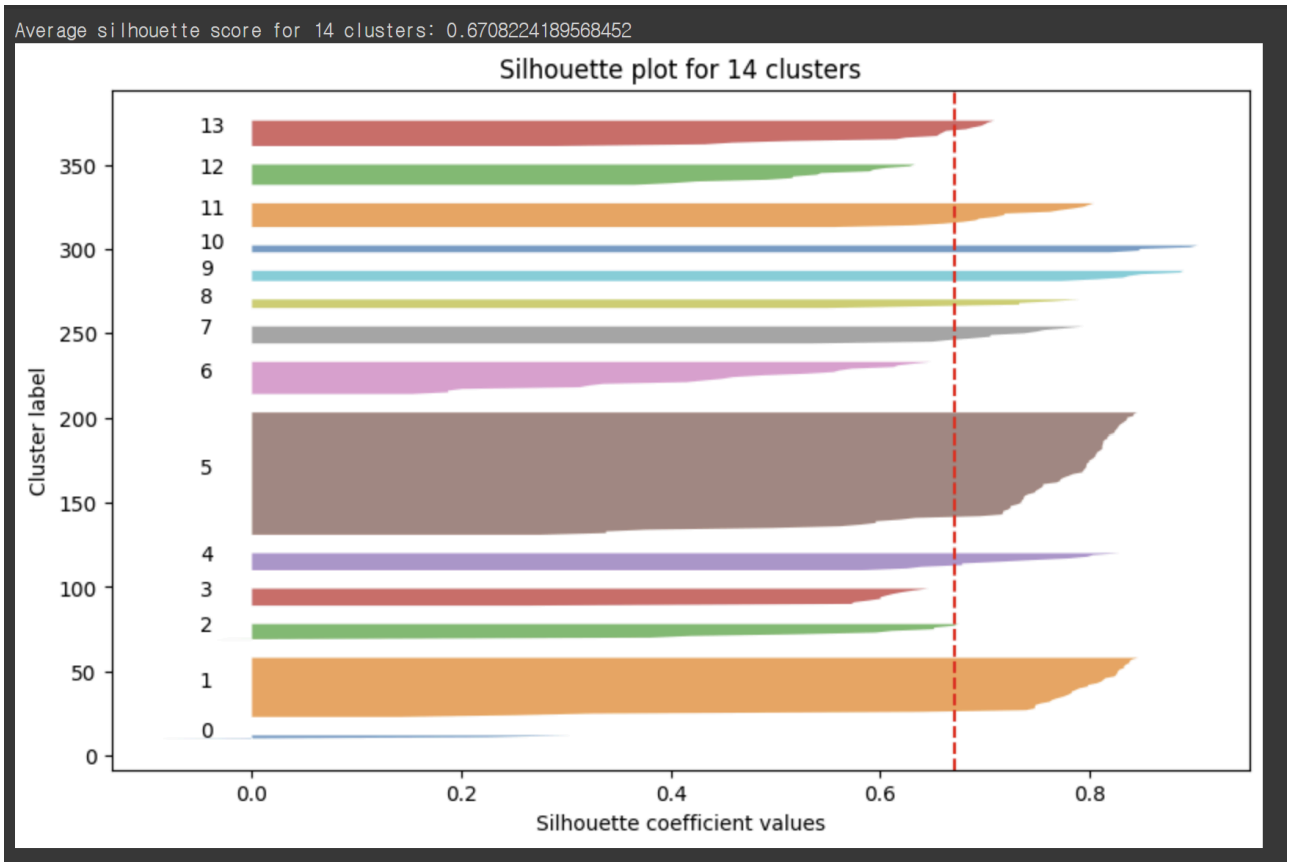
# 클러스터링 결과 시각화
plt.scatter(tfidf_matrix_sparse[:, 0].toarray(), tfidf_matrix_sparse[:, 1].toarray(), c=cluster_labels, cmap='viridis', s=50, alpha=0.5)
plt.show()
```



- hierarchical clustering 수행 결과 14개로 군집화된 모습과 Silhoutte-Score 좋은 모습

```
# 레이블 번호별 갯수 출력
print("Cluster label counts:")
for label, count in label_counts.items():
    print(f"Label {label}: {count}")
```

```
Cluster labels: [ 5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5 13
 13 13 13 13 13 13 13 13 13 13 13  5  5 13 13 13 13 13 13 13 13 13  8  8  8
  8  8  8  8  8  8  8  8  8  8  8  4  4  4  4  4  4  4  4  4  4  4 12
 12 12 12 12 12 12 12 12 12  0  0  0  0  0  0  3  3  3  3  3  3  6  6  6
  6  6  6  6  6  6  6  9  9  9  9  9  1  1  1  1  1  1  1  1  1  1  1
  1  1  7  7  7  7  7  7  7  7  7  7  7  7 10 10 10 10 10 10 10 10 10 10 10
 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 10 11 11
 11 11 11 11 11  2  2  2  2  2  2  2  2  2  2  2  2  2  2  2  5  5  5
  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5
  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5  5]
Cluster label counts:
Label 5: 73
Label 13: 20
Label 8: 15
Label 4: 11
Label 12: 10
Label 0: 6
Label 3: 6
Label 6: 11
Label 9: 5
Label 1: 13
Label 7: 11
Label 10: 33
Label 11: 7
Label 2: 16
```



💡 고민과 문제

- 기존에는 추천하는 시스템 생각은 없었지만 이후 과정에 추가하기로 결정되면서 클러스터가 형성되면 해당 클러스터를 대표하는 토큰들을 알고 이를 이용해 선호도 알고리즘에 활용을 해야 할 필요성이 생겼음

🛠️ 해결 방안

- 군집화가 수행되는 과정에서 핵심이 되었던 10개의 토픽과 해당 토픽의 지분을 저장해 사용하기로 결정

✅ 결과 및 성과

- 특정 클러스터 마다 핵심 토픽 10개가 저장된 모습

SQL | 인출된 모든 행: 26(0.08초)

	ID	TOPIC0	TOPIC1	TOPIC2	TOPIC3	TOPIC4	TOPIC5	TOPIC6	TOPIC7	TOPIC8	TOPIC9	TOPIC_VALUE0
1	0	헤즈볼라	레바논	네타냐후	휴전	연정	베이루트	이스라엘군	할레	리쿠드당	지상전	.12850827792343136
2	1	조우형	중수부	뉴스타파	부산저축은행	프레젠...	디넷	한상진	허경무	재판장	차명	.11280536123260082
3	2	김홍걸	여랑야랑	정회	동교동	박성준	권노갑	야바위	성혜	이서우	배기선	.09832006930466836
4	3	관리사	노량진	자습	정년	자격증	공인중개사	장년	크레바스	사무소장	사보	.19071099381673087
5	4	재석	모성	공포일	이재우	장애아	인당	보험자	양립	맞벌이	기권	.12136263125247386
6	5	육아휴직	체불	방지법	사업주	공포일	고용보험법	평등법	반의사불벌죄	한파	미숙아	.06480824691053372
7	6	빈집	소유주	구상권	창간	초장동	국제신문	이원준	철거	국비	지방세법	.381882418332534
8	7	장윤주	kcal	섬유질	두부	지방산	열량	포만감	연두부	설포라	미니멀	.17881066556040112
9	8	r	얇아지	휨	HBM	start	기가바이트	end	마이크론 ...	E	TSV	.09696141418841282
10	9	조재영	윤치영	·케이스탯	이지호	예은	특검법	순리	결자해지	표결	김건희	.1150854358460171
11	10	발각	항소심	대전지법	윗주머니	동남구	홍수현	봉지	형사부	마늘	범행	.08159719676811855
12	11	밍크고래	혼획	양양군	그물	어민	이현일	판장	허가서	어선	질식사	.23774984177892602
13	12	더위	세희	소나기	동풍	너울	한낮	우박	잔잔	불어오	밀려오	.10838899203011022
14	13	도루	홈런	론디포	맥과이어	오타니	ESPN	오타니 쇼헤이	공이	말린스	오타	.11498087064157689
15	14	격퇴	키이우	자포리자	크라마토...	주지사	방공	밝혔지	스비틀라나	군정청	페도로프	.08811842485749896
16	15	명씨	명태	김필	김동준	김영선	국민의당	하림	이모티콘	안철수	군	.19660164553244158
17	16	JTBS	배회	여고생	흉기	news	범행	행인	만취	일면식	순천시	.10489881019991475
18	17	복제품	룰루	로렐	민텔	Mintel	핑티	티몰	Nicole	Hal	pingti	.08709752522617406
19	18	김지우	성화선	농장주	영봉	조희경	식용	성운	동물자유연대	안락사	범수	.08173317075658096

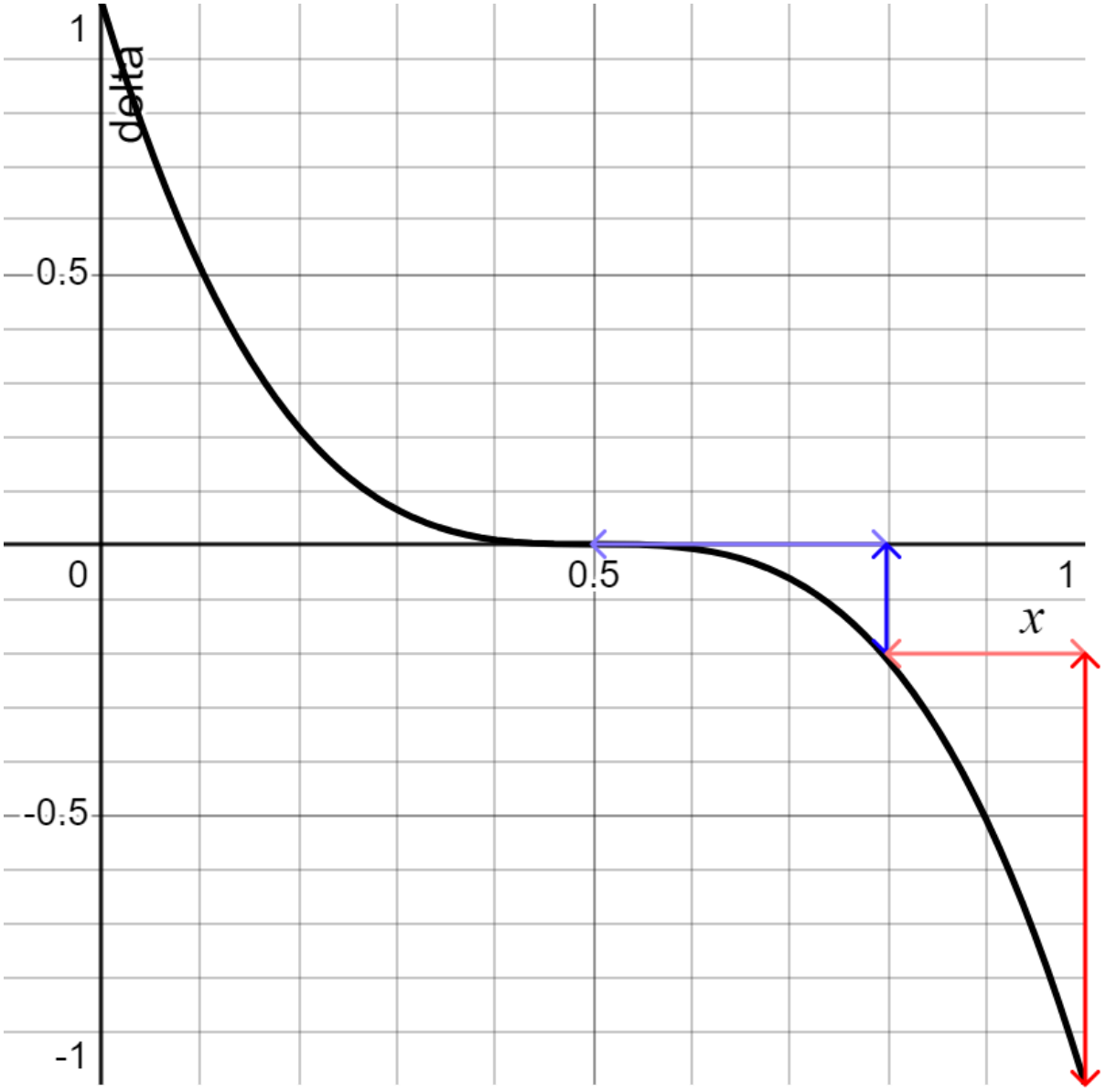
백엔드 시스템

💡 고민과 문제

- 백엔드의 기능들 중 균형잡힌 뉴스 소비를 지향하기 위해서는 결국 사용자의 선호도를 기반으로 잘 읽지 않는 분야 혹은 반대의 의견을 갖는 기사를 보여줘야 했고 이를 위한 추천 알고리즘의 설계가 필요했다
    - 문제는 결국 알고리즘이 잘 동작하는지 평가를 하기에 추천이란 시스템은 애매한 점이 문제였다

🛠️ 해결 방안

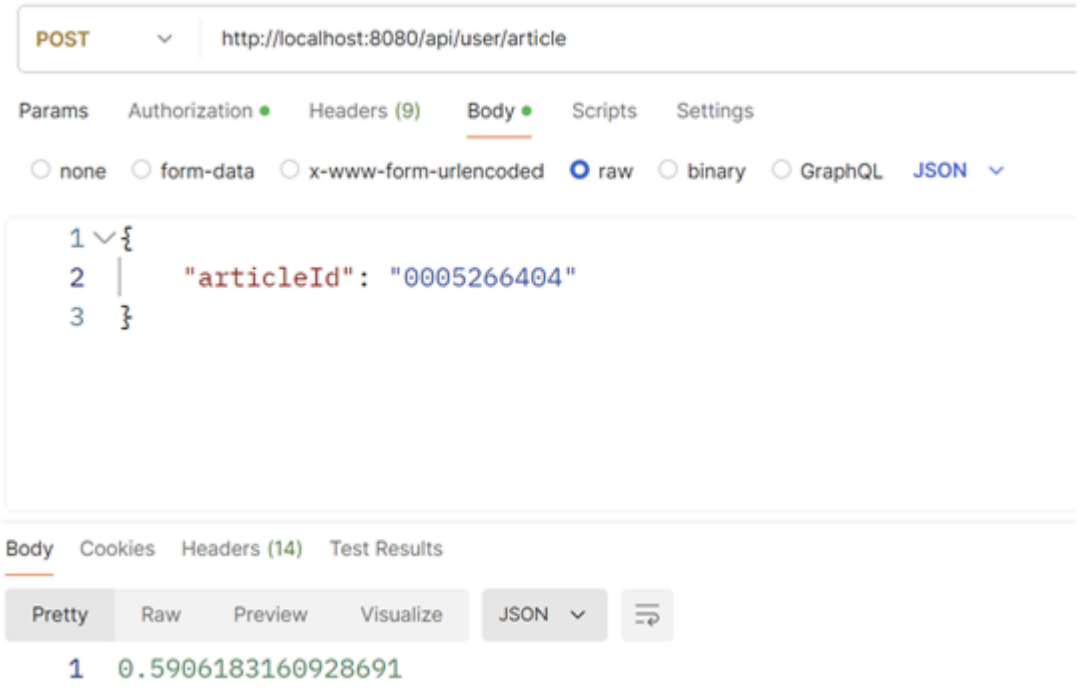
- 결국 수학적 이론을 활용 할 수 밖에 없었고 우리는 가정을 통해서 정상적으로 작동 함을 사전에 정의했다.
    - 방식은 랜덤으로 클러스터를 x개 뽑아오고 그 내부에서 y개씩 데이터를 뽑아오고 선호도 비교를 통해서 정렬을 수행한다
    - 이후 우리가 설정한 임계값 아래의 선호도를 갖는 데이터가 없다면 위의 가정을 반복하고 아니라면 해당 기사를 제공한다.
    - 만약 사용자의 인내심이 무한하다면 무한한 N의 시도 후에는 우리의 조건을 만족하는 기사를 추천 받게 될 것이다.
    - 사용자의 선호도를 보상하기 위한 보상 알고리즘에 활용한 아이디어



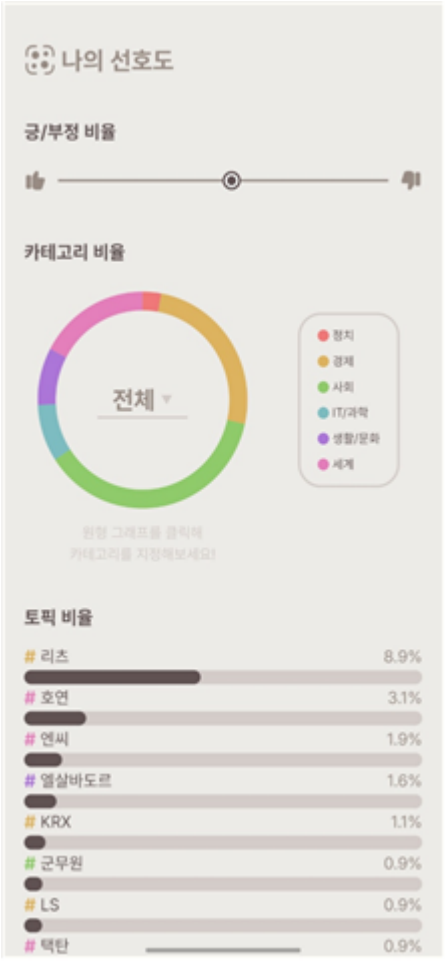
- 하지만 실제 사용자의 인내심은 유한하고 결국 실제 받는 경우에는 지연 시간이 적어야 하기 때문에 최대 반복 횟수를 설정해 현실에서는 빠른 시간안에 받도록 구현했다.

✅ 결과 및 성과

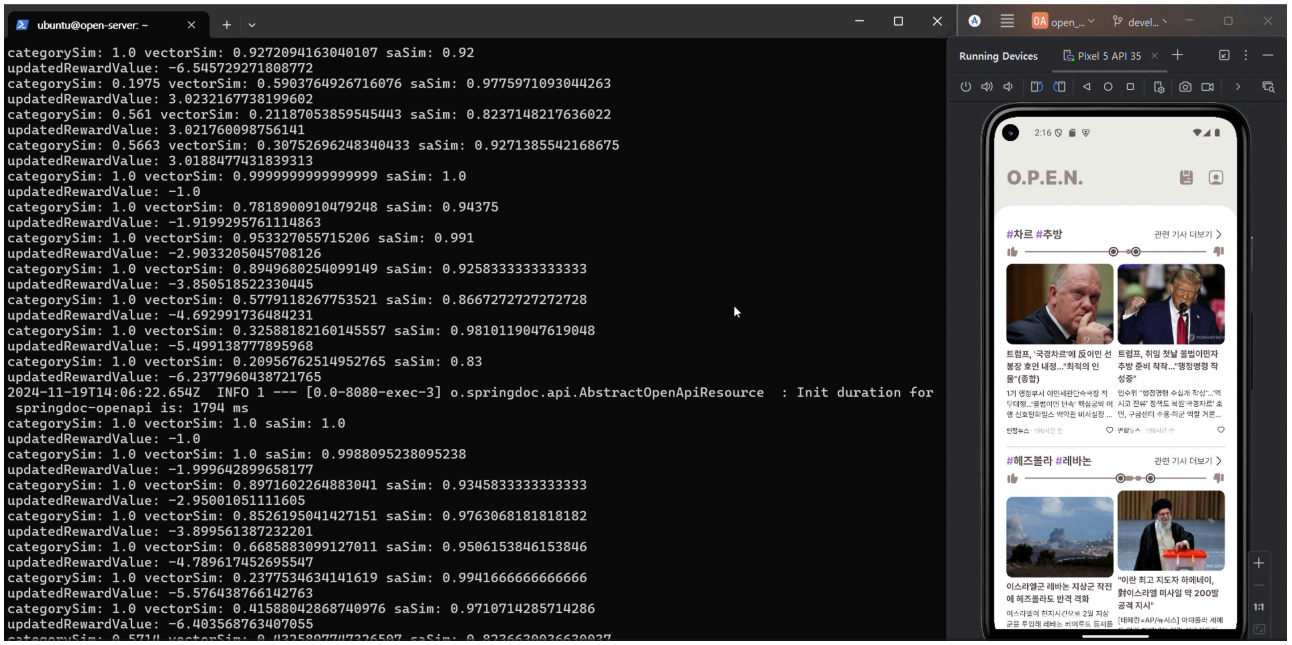
- 유저와 특정 클러스터간의 유사도 계산 결과



- 유저의 선호도 실제 모습



- 실제 유저 선호도 보상 내부 연산 모습



- 균형잡힌 기사 추천을 위한 API 동작

